

```
In [1]: import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
In [2]: # -----
# 1. Multi-Head Self Attention
# -----
class MultiHeadAttention(nn.Module):
    def __init__(self, embed_size, heads):
        super().__init__()
        self.embed_size = embed_size
        self.heads = heads
        self.head_dim = embed_size // heads

        assert self.head_dim * heads == embed_size, "Embed size must be divisible by the number of heads"

        self.values = nn.Linear(embed_size, embed_size)
        self.keys = nn.Linear(embed_size, embed_size)
        self.queries = nn.Linear(embed_size, embed_size)
        self.fc_out = nn.Linear(embed_size, embed_size)

    def forward(self, values, keys, query):
        N = query.shape[0]

        value_len, key_len, query_len = values.shape[1], keys.shape[1], query.shape[1]

        # Linear layers
        values = self.values(values)
        keys = self.keys(keys)
        queries = self.queries(query)

        # Split into heads
        values = values.reshape(N, value_len, self.heads, self.head_dim)
        keys = keys.reshape(N, key_len, self.heads, self.head_dim)
        queries = queries.reshape(N, query_len, self.heads, self.head_dim)

        # Energy (attention scores)
        energy = torch.einsum("nqhd,nkhd->nhqk", [queries, keys])

        attention = torch.softmax(energy / (self.embed_size ** (1/2)), dim=3)

        # Weighted sum
        out = torch.einsum("nhql,nlhd->nqhd", [attention, values])
        out = out.reshape(N, query_len, self.embed_size)

        return self.fc_out(out)
```

```
In [3]: # -----
# 2. Transformer Block
# -----
class TransformerBlock(nn.Module):
    def __init__(self, embed_size, heads, forward_expansion):
        super().__init__()
        self.attention = MultiHeadAttention(embed_size, heads)

        self.norm1 = nn.LayerNorm(embed_size)
        self.norm2 = nn.LayerNorm(embed_size)

        self.feed_forward = nn.Sequential(
            nn.Linear(embed_size, forward_expansion * embed_size),
```

```

        nn.ReLU(),
        nn.Linear(forward_expansion * embed_size, embed_size),
    )

    def forward(self, value, key, query):
        attention = self.attention(value, key, query)

        # Add & Norm
        x = self.norm1(attention + query)

        forward = self.feed_forward(x)

        # Add & Norm
        out = self.norm2(forward + x)

    return out

```

```

In [4]: # -----
# 3. Simple Transformer Model
# -----
class SimpleTransformer(nn.Module):
    def __init__(self, vocab_size, embed_size, heads, num_layers):
        super().__init__()

        self.embedding = nn.Embedding(vocab_size, embed_size)

        self.layers = nn.ModuleList(
            [TransformerBlock(embed_size, heads, forward_expansion=4)
             for _ in range(num_layers)]
        )

        self.fc_out = nn.Linear(embed_size, vocab_size)

    def forward(self, x):
        x = self.embedding(x)

        for layer in self.layers:
            x = layer(x, x, x)

        out = self.fc_out(x)
    return out

```

```

In [5]: # -----
# 4. Test the Model
# -----
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = SimpleTransformer(
    vocab_size=100,
    embed_size=64,
    heads=4,
    num_layers=2
).to(device)

# Dummy input (batch_size=2, sequence_length=5)
x = torch.randint(0, 100, (2, 5)).to(device)

output = model(x)

```

```
print("Input shape:", x.shape)
print("Output shape:", output.shape)
```

```
Input shape: torch.Size([2, 5])
Output shape: torch.Size([2, 5, 100])
```

In []: